Datazoic Agentic Platform

10-Stage Pipeline & Orchestration Workflow Specification

1. The Explicit 10-Stage State Machine

To eliminate the non-deterministic loops common in generic agent frameworks (such as raw ReAct loops), the Datazoic platform implements an explicit 10-stage LangGraph pipeline. Each turn executes through strict, observable stages with dedicated observability telemetry recorded into an immutable audit timeline.

Stage	Node Name	Core Operational Contract	Routing Logic
Stage 1	normalize_input	Sanitizes query, generates request UUID, initializes timeline	Proceeds to Stage 2
Stage 2	analyze_intent	Classifies domain intent, extracts entities, flags missing fields	Proceeds to Stage 3
Stage 3	select_registries	Scopes target namespaces (paypal, communication, general)	Proceeds to Stage 4
Stage 4	retrieve_tools	Hybrid Qdrant retrieval, returns bounded top-8 to 12 candidates	Proceeds to Stage 5
Stage 5	validate_candidates	Candidate Gate: validates parameter completeness or requests info	Missing $\to$ Stage 10; OK $\to$ Stage 6
Stage 6	plan_and_select	Formulates execution plan DAG selecting strictly from candidates	No Tool $\to$ Stage 10; Tool $\to$ Stage 7
Stage 7	policy_and_approval_gate	Evaluates risk levels, triggers HITL approval if mutating/financial	Pending $\to$ Stage 10; Approved $\to$ Stage 8
Stage 8	execute_tool	Validates parameters, dispatches via HTTP/SMTP/Mock engine	Proceeds to Stage 9
Stage 9	evaluate_result	Evaluates goal completion, binds Step 1 outputs into Step 2 inputs	Has Next $\to$ Stage 7; Done $\to$ Stage 10
Stage 10	compose_final_answer	Synthesizes user answer, records Gemini token usage in database	Transitions to END

2. Deep Dive: Candidate Gate & Clarification (Stage 5)

A critical vulnerability in standard LLM agents is guessing or fabricating missing inputs (e.g. inventing invoice amounts). In Stage 5, the **Candidate Gate** acts as a strict guardrail: if required parameters (such as recipient email or billing amount) cannot be extracted from conversational context, execution is intentionally halted, and a structured clarification prompt is presented to the user. The model is physically prevented from invoking downstream mutation tools until all prerequisites are validated.

3. Deep Dive: Dynamic Output-to-Input Chaining (Stage 9)

When executing composite multi-step requests, Stage 9 acts as the dynamic choreographic dispatcher. For example, consider the user query: 'What was my total sales volume last month? send report to dhivakarsa10@hotmail.com'.

1. **Step 1 Execution:** paypal_generate_settlement_report queries transaction records and returns {'total_sales_volume': {'value': '4250.75', 'currency_code': 'USD'}, 'total_items': 15, 'total_fee': {'value': '127.50'}}.
2. **Dynamic Synthesis:** In Stage 9, the evaluator intercepts this JSON payload, computes the net settled revenue ($4,123.25 USD), and dynamically formats a professional financial ledger summary.

3. **Argument Injection:** The evaluator injects this formatted ledger string directly into the body argument of Step 2 (send_email).
4. **Cyclical Turn Re-Entry:** The evaluator sets has_next_step = True, pointing the graph back to Stage 7 (Policy Gate) and Stage 8 (Execution), dispatching the live email via Google SMTP before reaching Stage 10.

4. End-to-End Visual Workflow Architecture

The flowchart below demonstrates the deterministic state routing, conditional branching gates, and evaluation loops:

```
+--------------------+ | 1. normalize_input | ===> Sanitizes query, creates request UUID
+--------------------+ | v +--------------------+ | 2. analyze_intent | ===> Classifies primary and
sub-intents (e.g. REPORTING + COMMUNICATION) +--------------------+ | v +--------------------+
|3.select_registries | ===> Scopes namespaces (paypal, communication, general) +--------------------+
| v +--------------------+ | 4. retrieve_tools | ===> Hybrid Qdrant Search: bounds top-8 to 12
candidate schemas +--------------------+ | v +--------------------+ Missing Data?
|5.validate_candidates| ----------------------> [ Stage 10: Clarification Prompt ]
+--------------------+ | (Validated) v +--------------------+ No Tool Needed? | 6. plan_and_select |
----------------------> [ Stage 10: Direct Answer ] +--------------------+ | (Planned Action) v
+--------------------+ Requires HITL? |7.policy_&_approval | ----------------------> [ Stage 10:
Render Approval Card ] +--------------------+ | (Approved) v +--------------------+ | 8. execute_tool |
===> Dispatches HTTP / Google SMTP / Sandbox Mock Engine +--------------------+ | v
+--------------------+ Step 1 of Multi-Step? | 9. evaluate_result | ----------------------+ (Injects
Output -> Input, Loops back) +--------------------+ | | (Goal Achieved) | v | +--------------------+ |
|10.compose_final_ans| <----------------------+ +--------------------+ | v [ END ]
```

5. Failure Modes & Self-Healing Resilience

- **Network Timeout Fallback:** If a live third-party endpoint fails or exceeds timeout thresholds, the execution engine records the error context in state, attempts exponential retries (up to 3 attempts), and gracefully falls back to deterministic simulation.
- **Embedding Quota Circuit Breaking:** If the Gemini embedding API responds with HTTP 429 Resource Exhausted, the tool registry automatically activates local deterministic hashing embeddings without interrupting workflow execution.
- **Strict Typo Sanitization:** Common user typing mistakes (e.g. user@domain,com) are normalized automatically during entity extraction prior to pipeline execution.